

Hackathon Problem Statement

Problem Statement: Asynchronous Video Processing System

Overview

You are required to design and build a **full-stack system** that handles **long-running video processing tasks asynchronously**. Users should be able to upload a video, request multiple output variants, track task progress, and download processed outputs once ready.

This problem is intentionally designed to evaluate **system design, asynchronous processing, state management, and explainability**, not just feature completion.

Mandatory Technology Stack

You must use the following:

- **Frontend:** React.js
- **Backend:** Node.js (any framework)
- **Database:** PostgreSQL

You are free to choose any additional libraries or architectural patterns.

Functional Requirements

1. Video Upload

- Accept video uploads with a maximum size of **200 MB**
- Supported input formats:
 - MP4
 - MOV
 - WebM
- Upload requests must return immediately after task creation

Uploading a file and processing a file must be treated as **separate responsibilities**.

2. Output Variants

For each uploaded video, users must be able to select one or more output variants. Each variant must be processed as an **independent task**.

Supported Output Formats

Variant	Container	Video Codec	Audio Codec
V1	MP4	H.264	AAC
V2	WebM	VP9	Opus

Resolution and Bitrate Profiles

Profile	Resolution	Target Bitrate
P1	480p	\~1 Mbps
P2	720p	\~2.5 Mbps
P3	1080p	\~5 Mbps

Exact media quality is **not evaluated**. These requirements exist to simulate long-running work.

3. Task Model

Each processing task must include:

- Unique task identifier
 - Reference to the uploaded video
 - Selected output format and profile
 - Task state
 - Timestamps for state transitions
 - Error information (if failed)
-

4. Task Lifecycle

Every task must transition through the following states:

- QUEUED
- PROCESSING
- COMPLETED
- FAILED

Task state transitions must be:

- Explicit
 - Persisted
 - Recoverable after refresh or restart
-

5. Asynchronous Processing Requirements

- Video processing must **not** occur inside HTTP request handlers
 - API endpoints must remain responsive while tasks are running
 - Multiple tasks may be processed concurrently
 - Failure of one task must not affect other tasks
-

Frontend Requirements

The frontend must reflect **backend state over time**.

It must:

- Display uploaded videos
- Display all associated tasks per video
- Show task state changes over time
- Persist accurate state across page reloads
- Provide download links for completed tasks
- Display failure states with error context

The UI must not rely on hardcoded or simulated state.

Persistence and Reliability Expectations

Your system should demonstrate that:

- Task state is stored durably
 - Refreshing the page does not reset task progress
 - Partial success is handled correctly
 - System stability is maintained during failures
-

Evaluation Criteria

Your submission will be evaluated on the following dimensions:

1. System Design and Architecture

- Clear separation of concerns
- Correct handling of long-running work
- Explicit task lifecycle modeling
- Awareness of failure and recovery scenarios

2. Backend Engineering

- Non-blocking request handling
- Clean API contracts
- Reasonable data modeling
- Explicit error handling

3. Frontend Engineering

- Accurate representation of backend state
- Time-aware UI behavior
- Reload safety
- Clear presentation of task progress and outcomes

4. Engineering Judgment and Explainability

- Ability to explain system design clearly
- Awareness of trade-offs
- Understanding of scale limitations
- Ability to reason about failures

Expected Level of Completion

You are **not expected** to fully complete every feature.

You **are expected** to:

- Correctly separate asynchronous work from API requests
- Correctly model tasks and task state
- Build a system that behaves correctly under long-running operations
- Clearly explain how your system works and where it could be improved

A partially complete system with strong design reasoning will score higher than a complete system with flawed architecture.

Submission Requirements

You must submit:

1. **GitHub Repository** containing frontend and backend code, with a README explaining setup and architecture
2. **Demo Video (5–8 minutes)** demonstrating the upload flow, task processing, and completed outputs
3. **Design Notes** explaining architectural decisions, failure handling, and scalability considerations

Final Note

This challenge evaluates how you **design, reason about, and explain systems that operate over time.**

Focus on correctness, clarity, and engineering judgment rather than feature count.